

AI Model File Formats — and What GGUF Is

A tour of how trained models get saved, shared, and run

Compiled July 30, 2026

A trained model's weights have to be serialized (saved to a file) before they can be stored, shared, or loaded for inference. Different formats optimize for different goals — training vs. inference, security, portability, or squeezing a model onto a laptop or phone. Below is an overview of the main formats in use today, followed by a closer look at GGUF specifically.

Common Model Formats

Format	Origin / Purpose	Typical Use
PyTorch .pt / .bin (pickle)	PyTorch's native serialization, based on Python's pickle module	Historic default for saving/loading models; being phased out because pickle can execute arbitrary code on load — a security risk when loading files from untrusted sources.
SafeTensors	Created by Hugging Face as a secure replacement for pickle-based files	Standard for storing and distributing model weights safely; supports memory-mapped, zero-copy loading; used for training and for direct GPU serving with engines like vLLM.
ONNX (Open Neural Network Exchange)	Backed by Microsoft, Meta, and others as a cross-framework standard	Train in one framework (e.g., PyTorch), export to ONNX, and run anywhere ONNX Runtime is supported — notably mobile NPUs (Apple Neural Engine, Qualcomm Hexagon), native Windows AI, and in-browser inference.
GGUF (GGML Unified Format)	Grew out of the llama.cpp project as GGML's successor format	The standard for running quantized LLMs locally on CPUs and consumer GPUs via llama.cpp, Ollama, and LM Studio. See below for detail.
MLX format	Apple's array/ML framework for Apple Silicon	Native format for the mlx-lm library; often delivers 20-40% higher decode throughput than GGUF/llama.cpp on Apple Silicon; also supports LoRA fine-tuning directly.
TensorRT-LLM engines	NVIDIA's inference-optimization toolkit	Compiles models into hardware-specific optimized engines for maximum throughput on NVIDIA GPUs in production serving.

What Is GGUF?

GGUF (GPT-Generated Unified Format) is a single-file binary format built for fast loading and efficient local inference. It was created for and is the native format of **llama.cpp**, and is now also supported by Ollama, LM Studio, and GPT4All. A model is normally trained and originally saved in SafeTensors, then converted to GGUF for portable, local, CPU/GPU-friendly deployment.

Why GGUF matters

- **Built-in quantization:** supports compressing weights from 16-/32-bit floats down to 8-bit, 4-bit, or even 2-bit integers (Q2 through Q8, plus various "k-quant" variants), shrinking model size by roughly 2-7x while keeping usable quality.
- **Single-file, self-contained:** one .gguf file bundles the weights, tokenizer configuration, and all metadata needed to run the model — no separate config files required.

- **CPU and mixed CPU/GPU inference:** optimized so large models can run on ordinary laptops and desktops, not just data-center GPUs — this is what makes running a model "locally" practical for most people.
- **Huge ecosystem:** as of 2026, well over 170,000 GGUF models are hosted on Hugging Face, making it the most widely available local-inference format.

GGUF vs. SafeTensors vs. ONNX — when to use which

- **Use SafeTensors** for training, fine-tuning, and direct GPU serving in production (e.g., with vLLM) — it's the format the model is usually trained and distributed in first.
- **Use GGUF** for running a model locally on a laptop, desktop, or CPU-heavy environment via llama.cpp, Ollama, or LM Studio, especially when you need quantization to fit the model in available memory.
- **Use ONNX** when you need to reach hardware GGUF doesn't specialize in — mobile NPUs, native Windows AI (DirectML/Windows ML), or in-browser inference via WebAssembly/WebGPU.
- **Use MLX** specifically on Apple Silicon Macs for the fastest local throughput, including fine-tuning in the same framework used for inference.
- These formats are complementary, not competing — the common real-world pattern is: train and store in SafeTensors, then convert to GGUF, ONNX, or MLX depending on where the model needs to run.

Converting a model to GGUF (typical workflow)

The usual path is: clone the llama.cpp repository, run its conversion script on a Hugging Face SafeTensors model to produce an initial full-precision (FP16) .gguf file, then run llama.cpp's quantization tool to produce a smaller quantized version (for example, a "Q4_K_M" 4-bit variant) sized to fit the target machine's memory.

Sources: ngxson.com "Common AI Model Formats"; Ertas AI "GGUF vs SafeTensors" and "GGUF vs ONNX" (2026); NEDNEX "Safetensors, CKPT, ONNX, GGUF, and Other Key AI Model Formats"; TinyWeights.dev "GGUF vs ONNX vs MLX"; Medium (Rohit Kushwaha) "Running GGUF and Safetensors Models."

This document is provided for general informational purposes only.